



Final Report

Group(22)

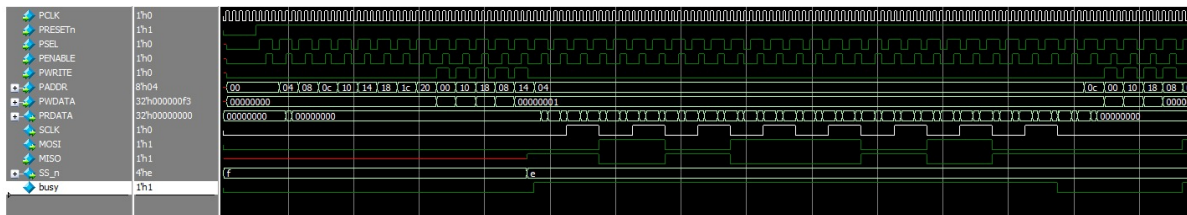
Name	ID	Sec
Ibrahim Mohamed Mohamed	2200769	ECE Sec 1
Youssef Mohamed Eissa	2200810	ECE Sec 1
Mohamed Essam Mohamed	2200835	ECE Sec 1
Miran mohamed	2200604	
Ehab tarek raafat	2201452	
Eman sherif sayed	2100721	
Nouran salah Ibrahim	2100532	

Tests

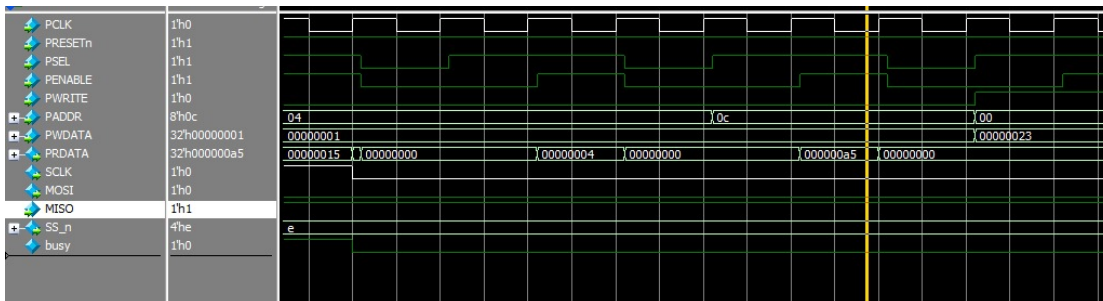
1)a) Sanity test

The test is executing a single, 1byte data transfer in a standard configuration (Mode 0, MSB-first) and verifying that the received data matches the expected value.

Data sent:

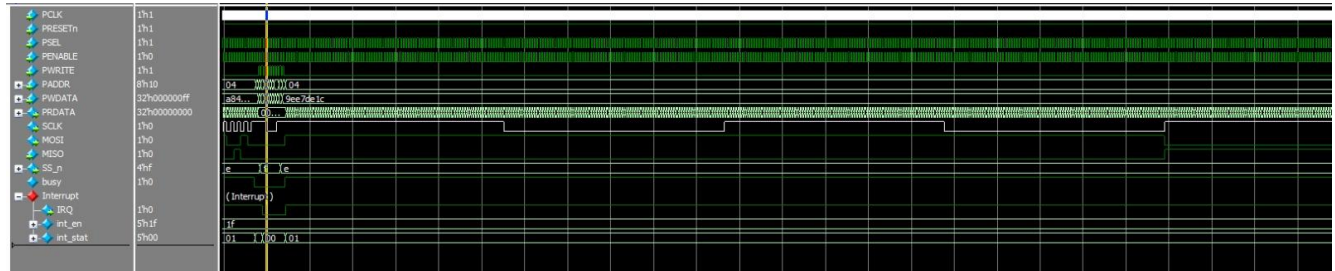
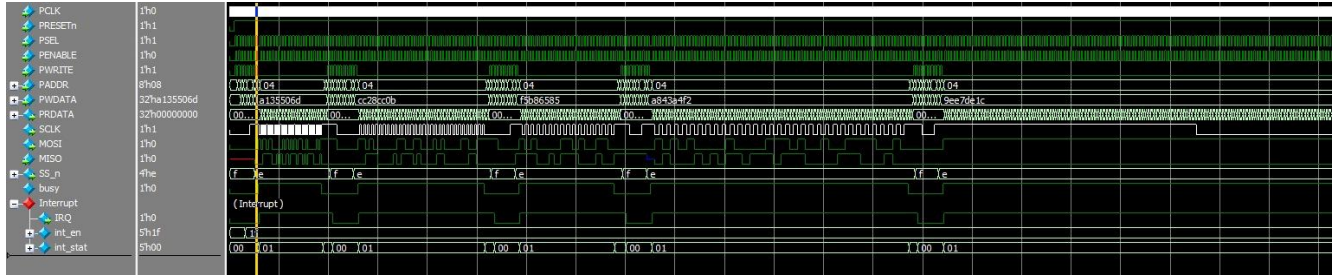


Data received :



```
# [INFO] Starting test: reg_access_test
# [INFO] reg_access_test: starting
# [INFO] Checking reserved offsets (0x24, 0x28)
# [INFO] reg_access_test: finished, errors=0
# [INFO] reg_access_test: finished, errors=0
# [TEST_PASSED] reg_access_test
# ** Note: $finish      : tb/tb_top.sv(162)
#   Time: 795 ns  Iteration: 2  Instance: /tb_top
```

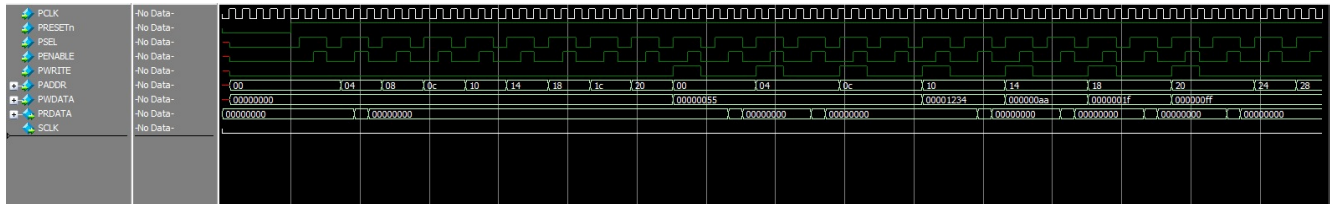
b) randomized sanity



```
# [INFO] Iter 16: mode=2 lsb=1 width=0 div=348 delay=83 tx=0x9186e031 lb=0
# [INFO] Iter 17: mode=0 lsb=1 width=2 div=498 delay=81 tx=0xec1dc4ba lb=0
# [INFO] Iter 18: mode=2 lsb=1 width=0 div=73 delay=216 tx=0xb912adf8 lb=1
# [INFO] Iter 19: mode=0 lsb=1 width=0 div=208 delay=177 tx=0xef5b4ba1 lb=1
# [INFO] Iter 20: mode=2 lsb=1 width=2 div=165 delay=135 tx=0xf908ccfc lb=1
# [INFO] Iter 21: mode=1 lsb=1 width=1 div=164 delay=33 tx=0xf077466a lb=0
# [INFO] Iter 22: mode=0 lsb=0 width=1 div=291 delay=175 tx=0xbe39ce10 lb=1
# [INFO] Iter 23: mode=3 lsb=1 width=2 div=149 delay=64 tx=0x225dl111 lb=0
# [INFO] Iter 24: mode=1 lsb=1 width=1 div=43 delay=204 tx=0x89f5ca4f lb=0
# [INFO] Iter 25: mode=0 lsb=1 width=0 div=354 delay=182 tx=0xfb3c9c73 lb=0
# [INFO] Iter 26: mode=1 lsb=1 width=1 div=262 delay=248 tx=0xba13bc18 lb=1
# [INFO] Iter 27: mode=1 lsb=1 width=2 div=218 delay=42 tx=0x696e304a lb=1
# [INFO] Iter 28: mode=0 lsb=0 width=2 div=477 delay=71 tx=0xefaab7fc lb=1
# [INFO] Iter 29: mode=3 lsb=1 width=0 div=345 delay=127 tx=0x594c6c26 lb=0
# [INFO] Iter 30: mode=3 lsb=1 width=0 div=93 delay=220 tx=0x51b407a6 lb=0
# [INFO] Iter 31: mode=1 lsb=1 width=2 div=433 delay=229 tx=0x780ea484 lb=0
# [INFO] Iter 32: mode=3 lsb=1 width=2 div=54 delay=29 tx=0x378d1a26 lb=1
# [INFO] Iter 33: mode=0 lsb=0 width=2 div=330 delay=178 tx=0x9dbc68d3 lb=1
# [INFO] Iter 34: mode=1 lsb=1 width=0 div=497 delay=90 tx=0x1af7a290 lb=0
# [INFO] Iter 35: mode=0 lsb=0 width=1 div=259 delay=251 tx=0x043c5d25 lb=1
# [INFO] Iter 36: mode=3 lsb=1 width=2 div=333 delay=73 tx=0x766d513c lb=1
# [INFO] Iter 37: mode=3 lsb=0 width=0 div=408 delay=178 tx=0x747eeba7 lb=1
# [INFO] Iter 38: mode=2 lsb=0 width=2 div=369 delay=234 tx=0x872e787f lb=0
# [INFO] Iter 39: mode=1 lsb=1 width=1 div=72 delay=103 tx=0x46896d8a lb=0
# [INFO] Iter 40: mode=2 lsb=0 width=1 div=265 delay=160 tx=0xab264d0f lb=1
# [INFO] Iter 41: mode=0 lsb=0 width=2 div=20 delay=243 tx=0x14c474e0 lb=0
# [INFO] Iter 42: mode=1 lsb=1 width=0 div=162 delay=83 tx=0x15f4d226 lb=0
# [INFO] Iter 43: mode=2 lsb=1 width=1 div=230 delay=1 tx=0x524ce0bf lb=1
# [INFO] Iter 44: mode=1 lsb=1 width=1 div=79 delay=80 tx=0x21468b7e lb=0
# [INFO] Iter 45: mode=3 lsb=0 width=0 div=431 delay=71 tx=0x0021f60c lb=1
# [INFO] Iter 46: mode=1 lsb=1 width=2 div=312 delay=162 tx=0x9543d75b lb=1
# [INFO] Iter 47: mode=0 lsb=1 width=0 div=208 delay=104 tx=0x7d3489fe lb=0
# [INFO] Iter 48: mode=1 lsb=0 width=0 div=193 delay=145 tx=0x14c6e2d9 lb=1
# [INFO] Iter 49: mode=1 lsb=1 width=0 div=4 delay=97 tx=0xede91ac4 lb=0
# [INFO] randomized_sanity_test finished, errors=0
# [INFO] randomized_sanity_test: finished, errors=0
# [TEST_PASSED] randomized_sanity_test
# ** Note: $finish      : tb/tb_top.sv(162)
# Time: 14902355 ns Iteration: 3 Instance: /tb_top
```


2) reg access test

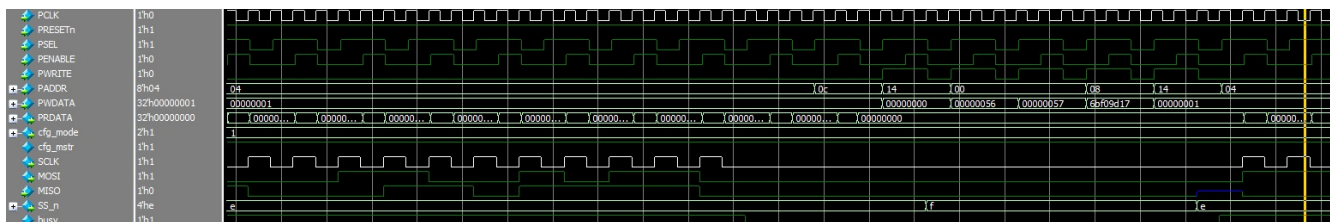
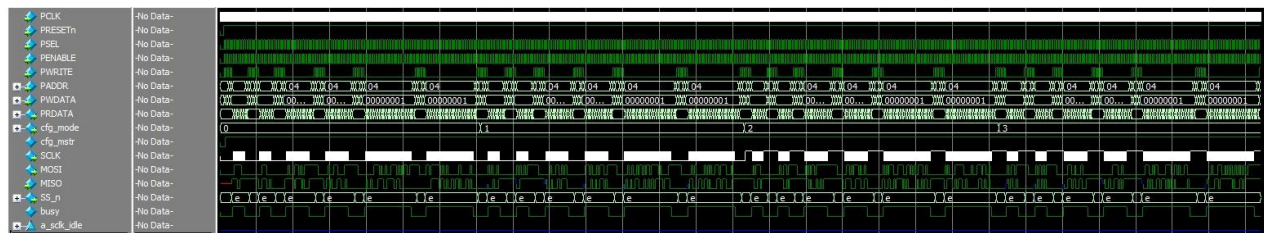
The test ensures that the hardware registers can be correctly addressed, that they hold the expected default values after reset, and that Read/Write (R/W) operations function as intended without corrupting data or allowing access to reserved memory space.



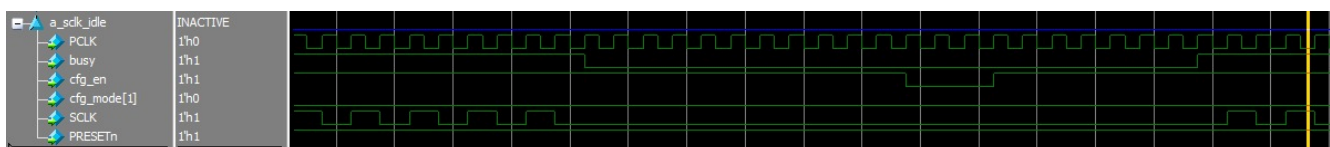
```
# [INFO] Starting test: reg_access_test
# [INFO] reg_access_test: starting
# [INFO] Checking reserved offsets (0x24, 0x28)
# [INFO] reg_access_test: finished, errors=0
# [INFO] reg_access_test: finished, errors=0
# [TEST_PASSED] reg_access_test
# ** Note: $finish      : tb/tb_top.sv(162)
# Time: 795 ns Iteration: 2 Instance: /tb_top
```

3) mode coverage test

The test focuses on cycling through all combinations of hardware configurations to ensure the design operates correctly across its all supported modes



Sclk idle assertion:



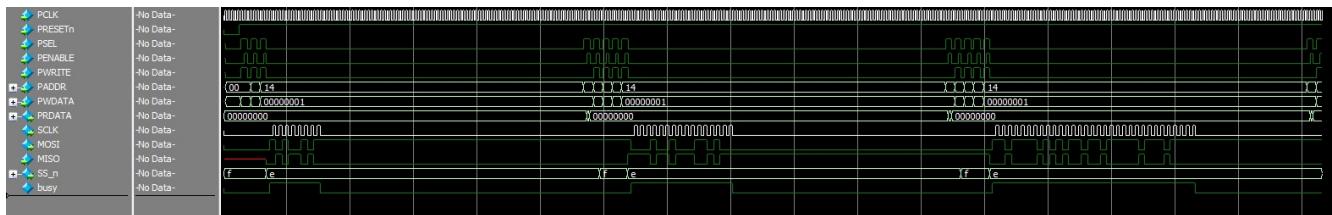
```

# Loading work_spi_slave_bram(test)
# [INFO] Starting test: mode_coverage_test
# [INFO] mode_coverage_test: starting
# [INFO] Testing Mode=0, Width=8b, LSB=0
# [INFO] Testing Mode=0, Width=8b, LSB=1
# [INFO] Testing Mode=0, Width=16b, LSB=0
# [INFO] Testing Mode=0, Width=16b, LSB=1
# [INFO] Testing Mode=0, Width=32b, LSB=0
# [INFO] Testing Mode=0, Width=32b, LSB=1
# [INFO] Testing Mode=1, Width=8b, LSB=0
# [INFO] Testing Mode=1, Width=8b, LSB=1
# [INFO] Testing Mode=1, Width=16b, LSB=0
# [INFO] Testing Mode=1, Width=16b, LSB=1
# [INFO] Testing Mode=1, Width=32b, LSB=0
# [INFO] Testing Mode=1, Width=32b, LSB=1
# [INFO] Testing Mode=2, Width=8b, LSB=0
# [INFO] Testing Mode=2, Width=8b, LSB=1
# [INFO] Testing Mode=2, Width=16b, LSB=0
# [INFO] Testing Mode=2, Width=16b, LSB=1
# [INFO] Testing Mode=2, Width=32b, LSB=0
# [INFO] Testing Mode=2, Width=32b, LSB=1
# [INFO] Testing Mode=3, Width=8b, LSB=0
# [INFO] Testing Mode=3, Width=8b, LSB=1
# [INFO] Testing Mode=3, Width=16b, LSB=0
# [INFO] Testing Mode=3, Width=16b, LSB=1
# [INFO] Testing Mode=3, Width=32b, LSB=0
# [INFO] Testing Mode=3, Width=32b, LSB=1
# [INFO] mode_coverage_test: finished, errors=0
# [INFO] mode_coverage_test: finished, errors=0
# [TEST_PASSED] mode_coverage_test

```

4) width coverage test

Verify the transfer with all the data widths 8_bits ,16_bits and 32_bits



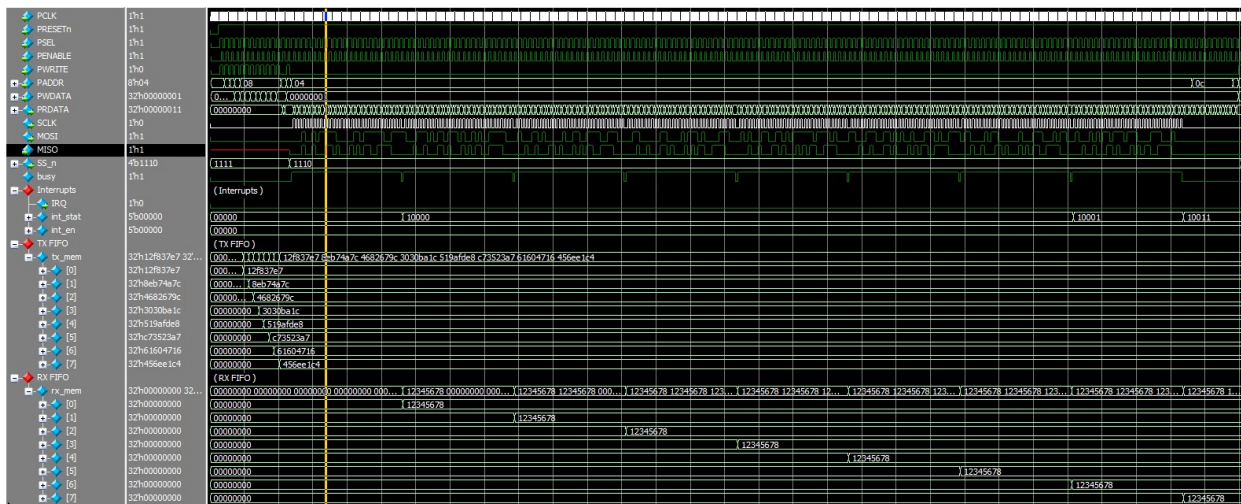
```

# [INFO] Starting test: width_coverage_test
# [INFO] width_coverage_test: starting
# [INFO] width_coverage_test: finished, errors=0
# [INFO] width_coverage_test: finished, errors=0
# [TEST_PASSED] width_coverage_test
# ** Note: $finish : tb/tb_top.sv(162)
# Time: 3465 ns Iteration: 3 Instance: /tb_top

```

5) fifo stress test

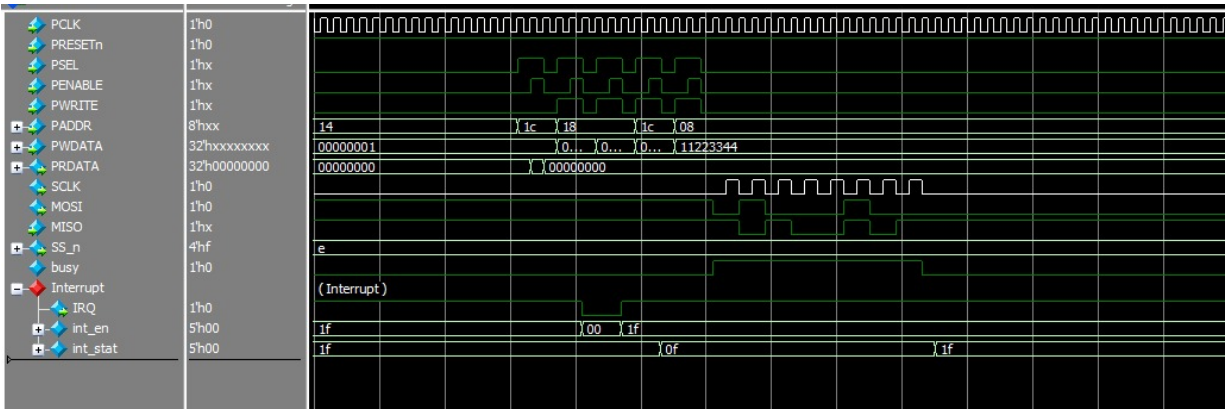
The test aims to saturate the hardware by filling the TX FIFO to its maximum capacity and then triggering a continuous transmission that fills the RX FIFO



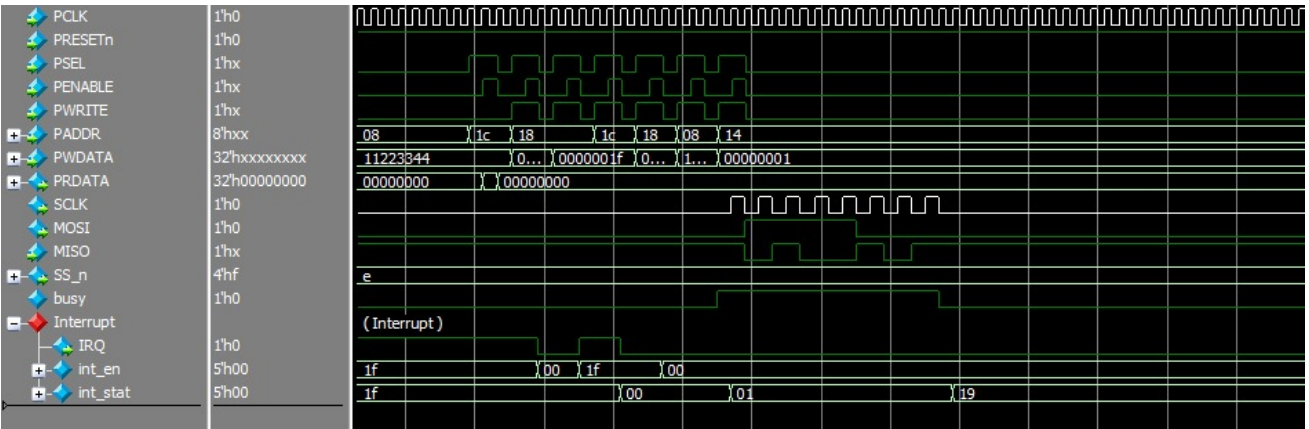
100



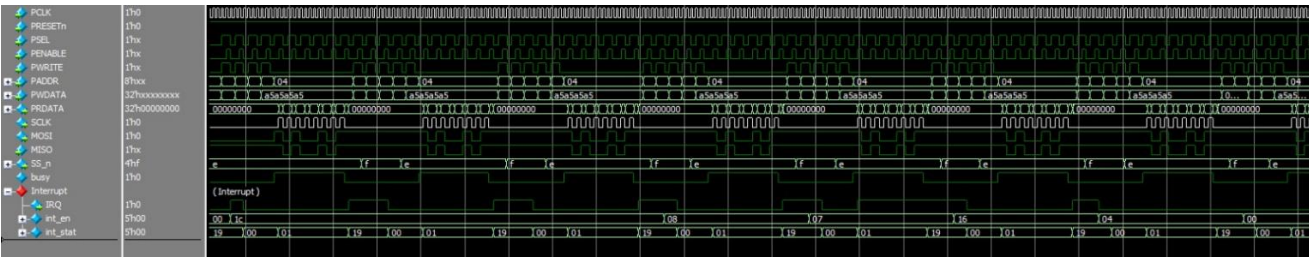
We are verifying that the hardware correctly handles dynamically disabling (masking) interrupts mid-flight, and that rewriting configuration data or changing the operation mode works seamlessly right before initiating an isolated single-word transaction.



This test specifically validates how the interrupt engine handles dynamic masking (`int_en`) immediately followed by a Write-1-to-Clear (W1C) status reset (`int_stat`), and ensures that a subsequent transmission accurately triggers the next isolated status update.

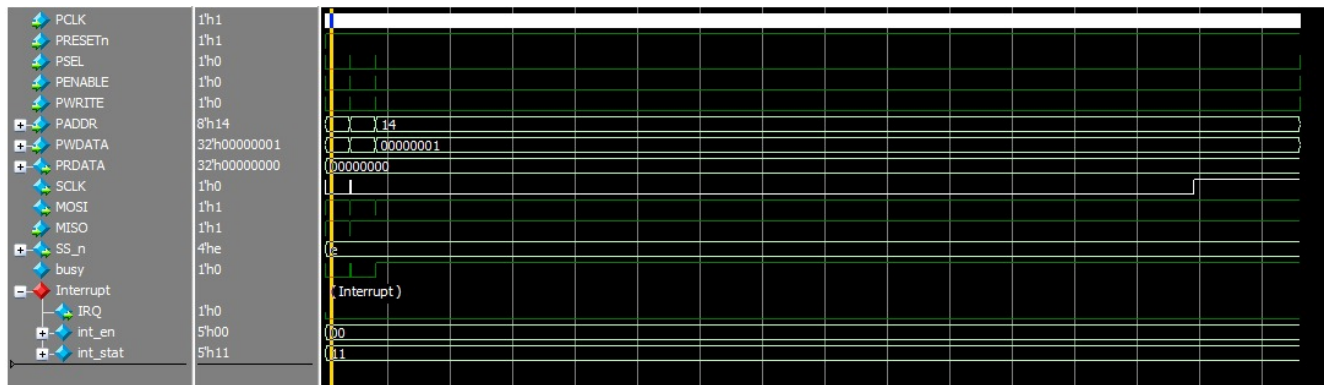
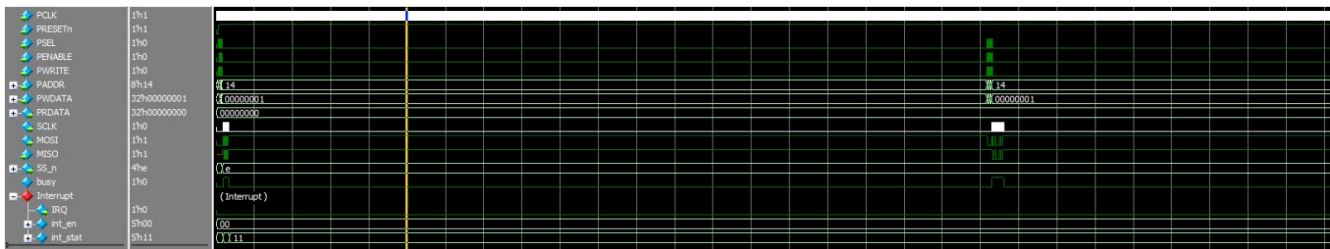


We are verifying that the APB-to-SPI core can handle repeated, discrete single-word cycles while cleanly asserting and de-asserting the protocol control lines every single time.



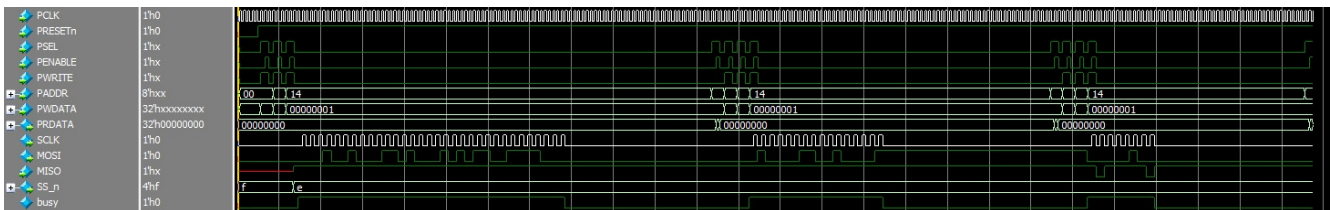
7) Clk division corner test

test for the clock divider logic. It specifically focuses on corner cases the extreme minimum and maximum values to ensure the SCLK generation is stable across the entire supported range



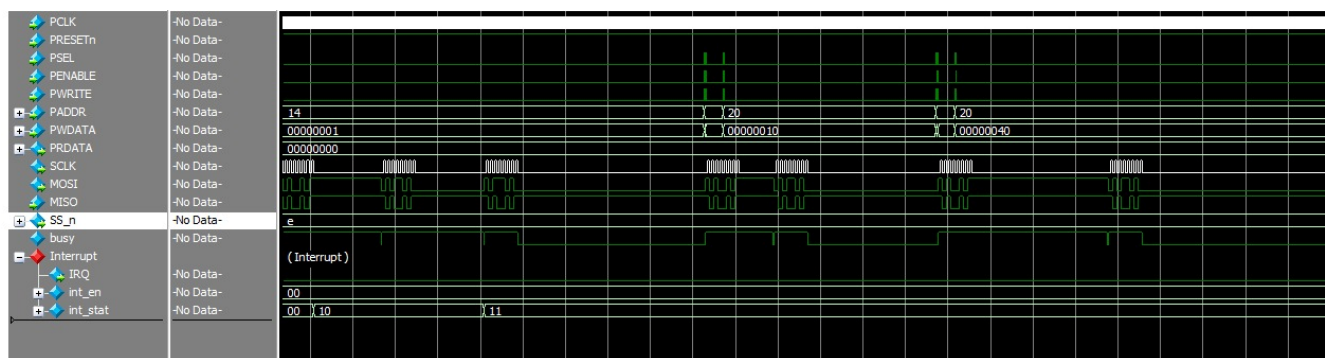
8) loopback test

It verifies that data sent out by the transmitter is correctly routed back and received by the same controller when "loopback mode" is enabled.



9) delay transfer test

verifies the SPI controller's ability to insert programmed idle periods between data transfers. This is typically used to meet the timing requirements of slower slave devices that need time to process data between words.



10) error injection test

TX write when full, RX read when empty, illegal width encoding, access to reserved offsets

